论文解读 · arXiv 2611.12885 · cs.CV · 2026-11-09

World In-Context Learning for Agentic Robot Manipulation

让机器人在“上下文”里学会操作：一次演示 + 仿真排练，零梯度更新

Heng Fang, Zehui Guo, Yuehan Zhang, Mingyang Du, Dingkang Liang, Xiang Bai
华中科技大学 · ArcheOne AI 代码： github.com/ArcheOne-AI/RoboScript

TL;DR

一句话总结：把技能存进可读的上下文，比存进权重更能跨分布

冻结的基础模型从 **1 条演示**出发，在仿真器里**排练**任务，把世界反馈写成语义化的 **Task Memory**；不做任何梯度更新，就在 RoboTwin 2.0 全部 50 个任务上超过用 50 条演示训练的所有动作策略。

1

条演示 / 每任务

对比基线 50 条

0

次梯度更新

权重全程冻结

83.4%

50 任务平均成功率

最强训练策略 61.7%

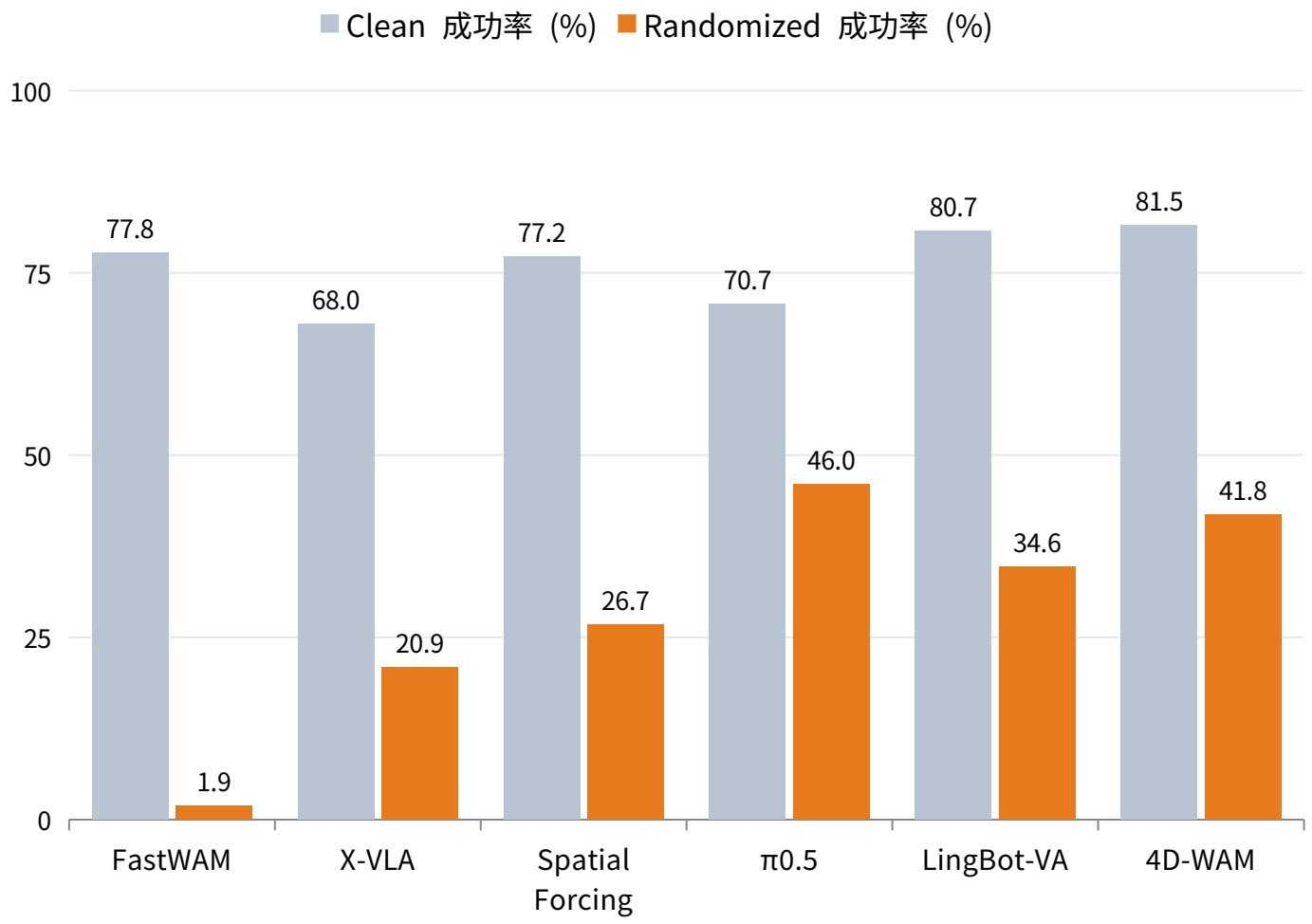
99.5%

随机化保持率

训练策略最高 65.1%

MOTIVATION

问题：训练出的策略把技能和训练分布绑死，一换分布就崩



FastWAM

77.8% → 1.9%

clean → domain randomization

权重学习

技能写进网络参数，与训练分布耦合；新任务 / 新分布都要重新采数据、重新训练。

传统 ICL

演示录自别的场景，没有当前世界的反馈； GPT-6 Astra 换布局演示后并无提升。

启发： coding agent

在终端里运行代码、读输出来熟悉新仓库 → 机器人缺少的正是这个反馈回路。

来源： Table 1 （训练策略均用每任务 50 条 clean 演示）

KEY INSIGHT

核心洞察：上下文里学到的是语义，因此可以跨世界迁移

Learning in Weights

梯度训练，技能在 θ 中

绑定训练域

In-Context Learning

演示放进上下文 C

绑定所示场景

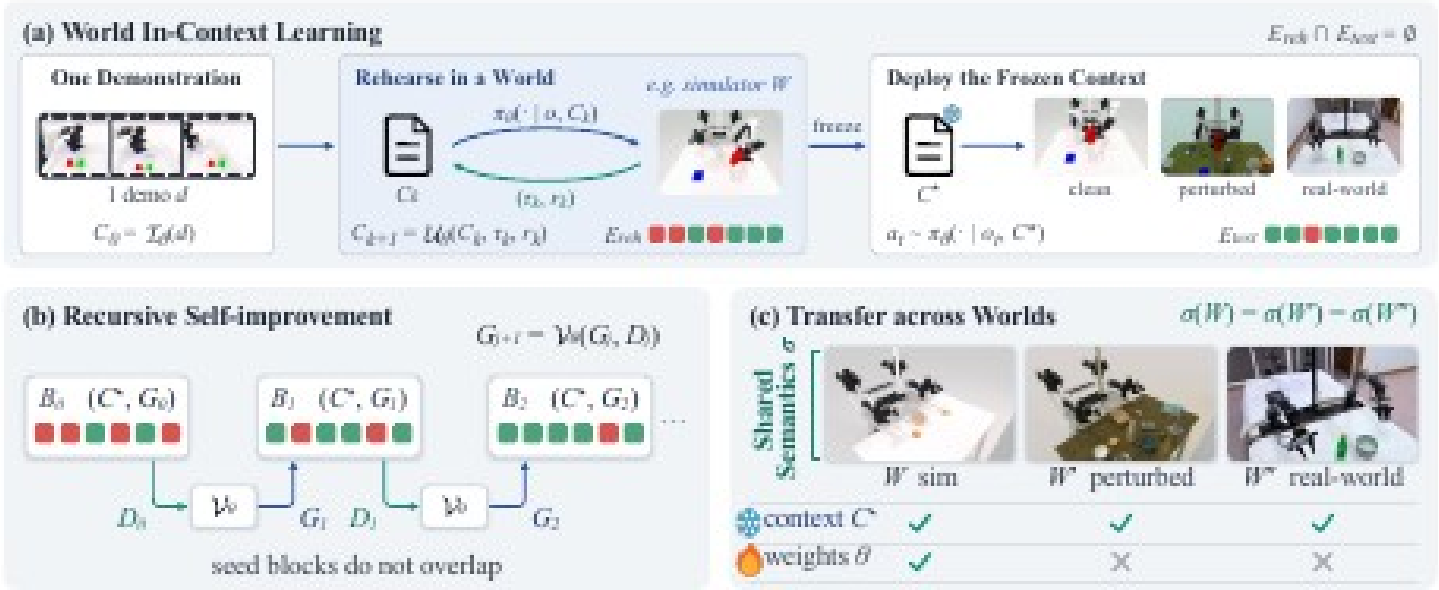
World In-Context Learning

在世界中排练，技能写入 C

绑定任务语义 → 可迁移

Task Memory 记录物体角色、任务步骤与教训，而非视觉外观（Fig.1b：仿真器 ≈ 终端， Task Memory ≈ AGENTS.md ）

WICL 形式化： ICL + 世界反馈，权重冻结，只有上下文在变



① 排练与修订

$C_0 = I_\theta(d)$

$(\tau_k, r_k) \sim W(\pi_\theta(\cdot|o, C_k), e_k)$

$C_{k+1} = U_\theta(C_k, \tau_k, r_k)$

e_k ∈ E_reh ，同一冻结模型做初始化与更新

② 冻结部署

$a_t \sim \pi_\theta(\cdot \mid o_t, C^*)$

$e \in E_{test}$, $E_{reh} \cap E_{test} = \varnothing$

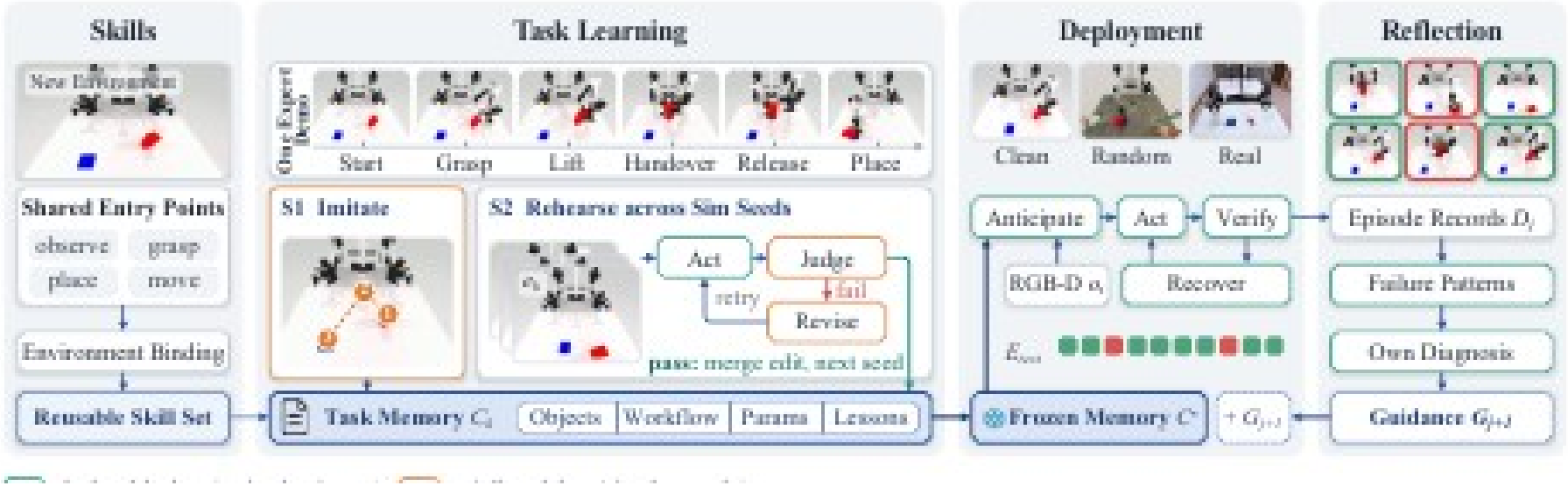
测试实例从未排练 → 测泛化而非记忆

③ 递归自我改进

$G_{j+1} = V_\theta(G_j, D_j)$

在不重叠评测块上反思；记录只含部署可见信息与成败

RoboScript： 技能构建 → 一次演示学习 → 冻结部署 → 反思改进



1 Skills

共享入口 + 环境绑定； coding agent 为演示中解释不了的片段写技能，过闸门后冻结

2 Task Learning

S1 模仿：读演示写记忆并练到稳定

S2 排练：新 seed 只算首次尝试，修订仅在让该实例成功时合并

3 Deployment

场景预判（角色映射、选臂、可行性） + 不重置的失败恢复 + 对照演示终态的完成检查

4 Reflection

读部署记录（仅可见信息 + 成败），自行诊断失败模式，写下一块的 Guidance

Task Memory：一份只含语义与相对几何的 Markdown，由 linter 把关

place_object_basket • 冻结 Task Memory （节选， GPT-6.1-Sol ）

INSTRUCTION	Grab the bright sleek toycar, drop it in the basket ...
OBJECTS	toycar: the object to drop into the container ... basket: receives the object and is then moved ...
WORKFLOW	1 prepare → 2 grasp → 3 handover → 4 lift → 5 drop → 6 retreat → 7 grasp_basket → 8 move_basket
PARAMETERS	toycar_drop = place_spec(target=basket top_center, frame=target, ...) + 19 more specs
LESSONS	L2 ... keep the arm selected for the object through retreat ...
NOTES	The demonstration ends with the loaded basket still grasped ...

相对规格，而非位姿

GraspSpec / PlaceSpec 以感知到的物体部件为参照，如“从左侧接近把手、抓中点”，执行时在当前几何上求解，并可镜像给另一只手。

Memory linter

拒绝坐标、绝对位置、与机器人的距离、仿真器名、评测术语、实例 ID 、固定手臂分配。

排练写进记忆

此例中 stage 3 （换手交接）和两条 lessons 都是排练阶段加入的。

动作回执 receipts

每个动作返回指令位姿、实际位姿、夹爪开度、是否持物——来自本体感知与规划器，真机同样可得。

数据隔离：执行者只看真机可得输入，特权信息只做监督

所有模型调用经统一 Model Gateway，换基础模型只需改配置； Agent plane 只走可部署接口 (D)，只有 Control plane 能访问特权数据 (P)；方法、基准、本体都是可替换插件。

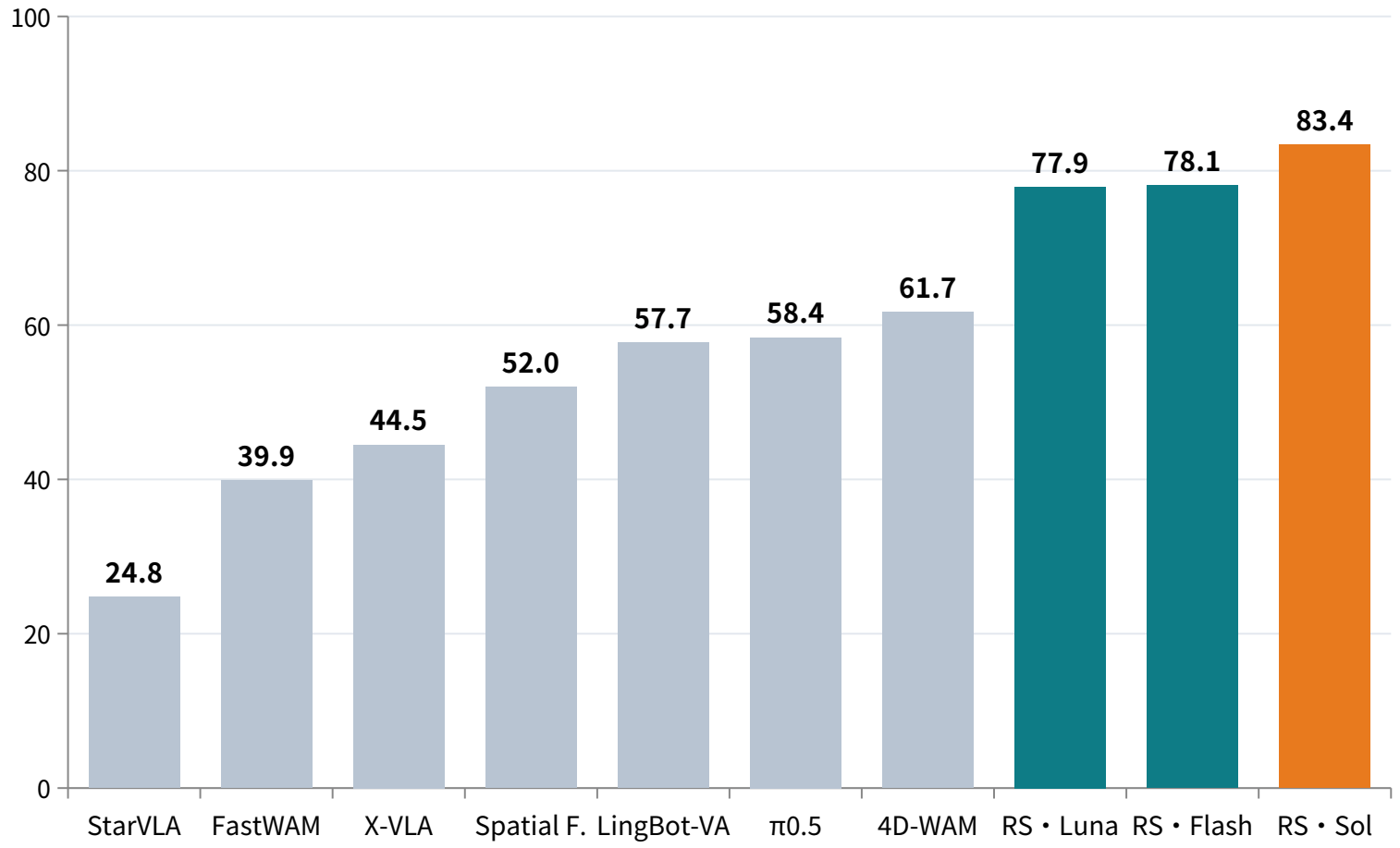
阶段	角色	数据层	可改记忆
S1	演示阅读者	**P**	是
S1	执行者	**P （演示仅作提示）**	否
S1	审阅者	**P**	是
S2	执行者	**D**	否
S2	审阅者	**D + 真值诊断**	缓存副本
部署	执行者	**D**	否

表：各角色可用数据（据 Table 5 整理）

隔离检验：去掉全部特权监督（演示真值状态 + 失败真值诊断）后， 10 任务均值 **86.5%** vs 完整方法 **86.0%**—— 性能并非来自特权信息。

主结果： 1 条演示、零梯度，平均 83.4% ，领先最强训练策略 21.7 分

Clean 与 Rand 平均成功率（%） 灰：训练动作策略（50 demos） 青 / 橙： RoboScript （1 demo）



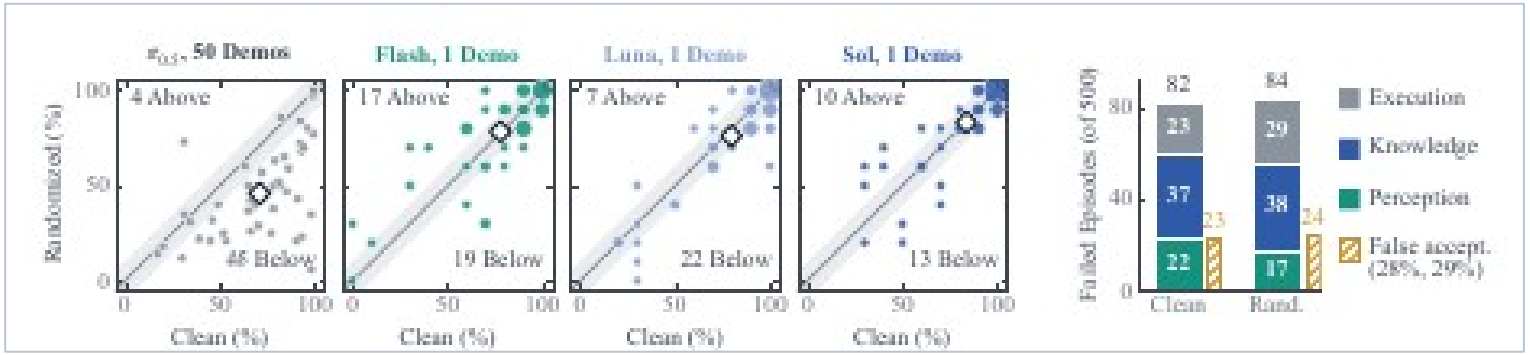
模型	Clean	Rand.	保持率
GPT-6.1-Sol	**83.6**	**83.2**	**99.5%**
GPT-5.6-Luna	79.8	76.0	95.2%
DeepSeek-V4.1-Flash	77.8	78.4	100.8%
4D-WAM （最强基线）	81.5	41.8	51.3%

要点

- clean 下与 4D-WAM 相当（83.6 vs 81.5），差距主要来自随机化（83.2 vs 41.8）
- Sol 95% Wilson 区间 81.0–85.6
- 三个模型同一套代码 / 提示 / 协议，均超过全部训练策略

来源： Table 1 （RoboScript 每任务每设置 10 episodes；基线 100 episodes）

只在 clean 场景排练，随机化下几乎不掉点：失败是物理性的



(a) 每任务 clean vs 随机化： π0.5 有 45 个任务落在对角线下方， RoboScript 三个模型散布在两侧。 (b) Sol 的失败按根因统计。

解读：随机化桌高让动作更难（执行失败增加），但不影响“知道什么”； false acceptance 稳定在 28% / 29% 。

95.2–100.8%

三个模型的随机化保持率；训练策略最高 65.1%

37 → 38

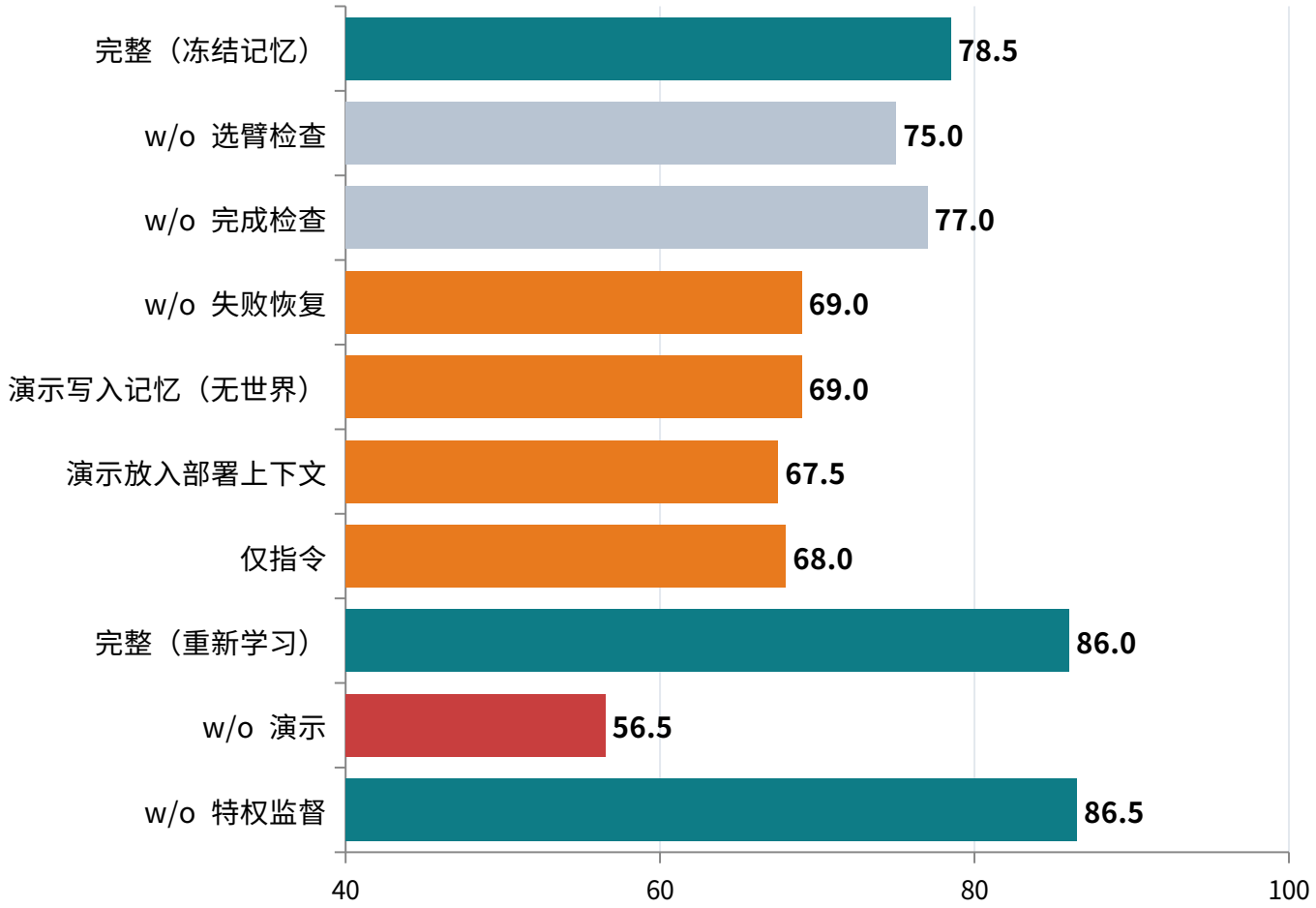
Sol 知识类失败（ clean → rand ）几乎不变；感知 22→17 ，执行 23→29

78%

可变物体任务中，用未见物体实例的 episode 占比，成功率无显著差异（CMH p=0.97）

消融：演示与世界缺一不可——没有世界反馈，演示几乎没用

平均成功率（%， clean 与 rand 均值）；各组与本组“完整”配对比较



世界不可少

同一演示：无世界交互 69.0% vs 有排练 78.5%（p=0.003）；三种“无世界”变体相差 ≤ 1.5 分。

演示是锚点

去掉演示从零排练： 86.0% → 56.5% ，降幅最大，部分任务不收敛。

排练修正细节

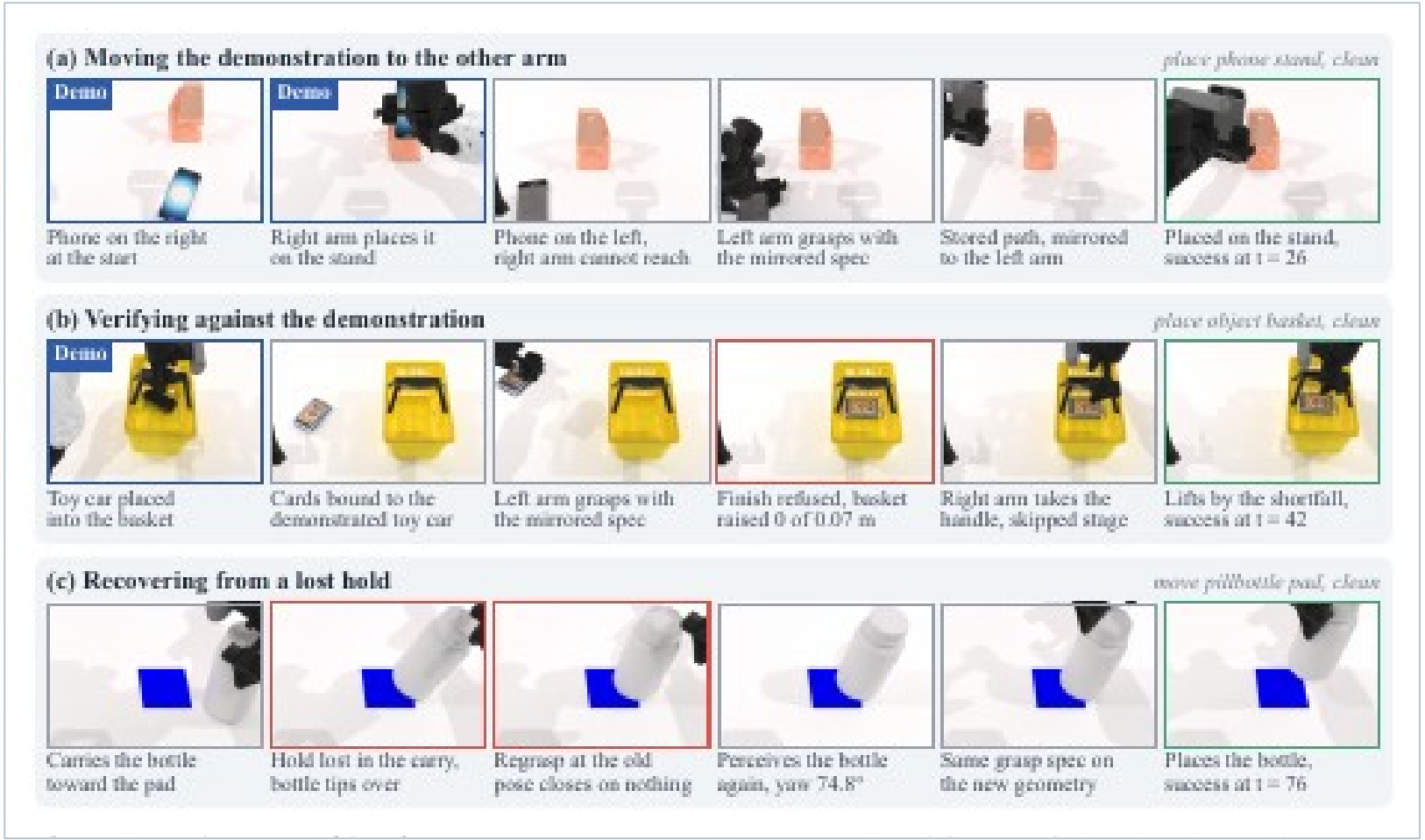
11 个难任务上排练带来 +8.4 分，随机化下 +13.3 分（p=0.034）。

部署组件

去掉失败恢复跌幅最大（78.5 → 69.0）；特权监督去掉不影响（86.5）。

来源： Table 2 （GPT-6.1-Sol， 10 任务 × 10 episodes × 2 设置，配对比较）

部署行为：镜像换臂、拒绝过早结束、掉落后重新抓取



(a) 选臂检查

物体在另一侧、演示手臂够不到时，用镜像后的规格由另一只手抓取；成功抓取中约一半换了臂，成功率相当（93% / 95%）。

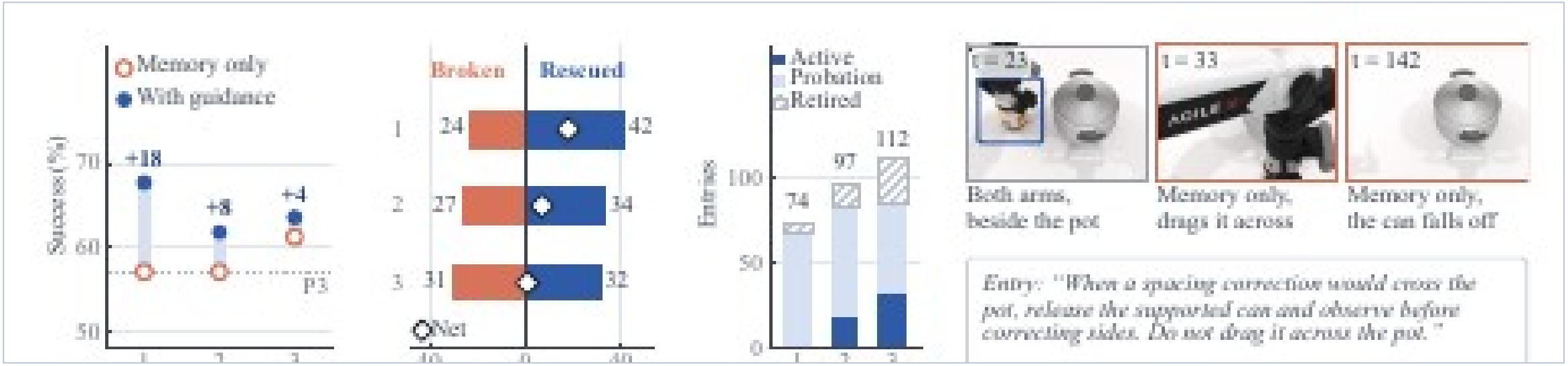
(b) 完成检查

终态与演示不符（篮子未被抬起）时拒绝结束，回到被跳过的阶段。

(c) 失败恢复

搬运中掉落后不重置环境，重新感知瓶子姿态，用同一抓取规格在新几何上重抓并完成。

自我改进：权重冻结，反思写出的 Guidance 每轮都有净收益



+18 / +8 / +4

17 个弱任务上三轮的净增益
（救回 − 破坏的 episode ）

+18 / +7 / +1

全部 50 任务上的净增益
收益随轮次快速衰减

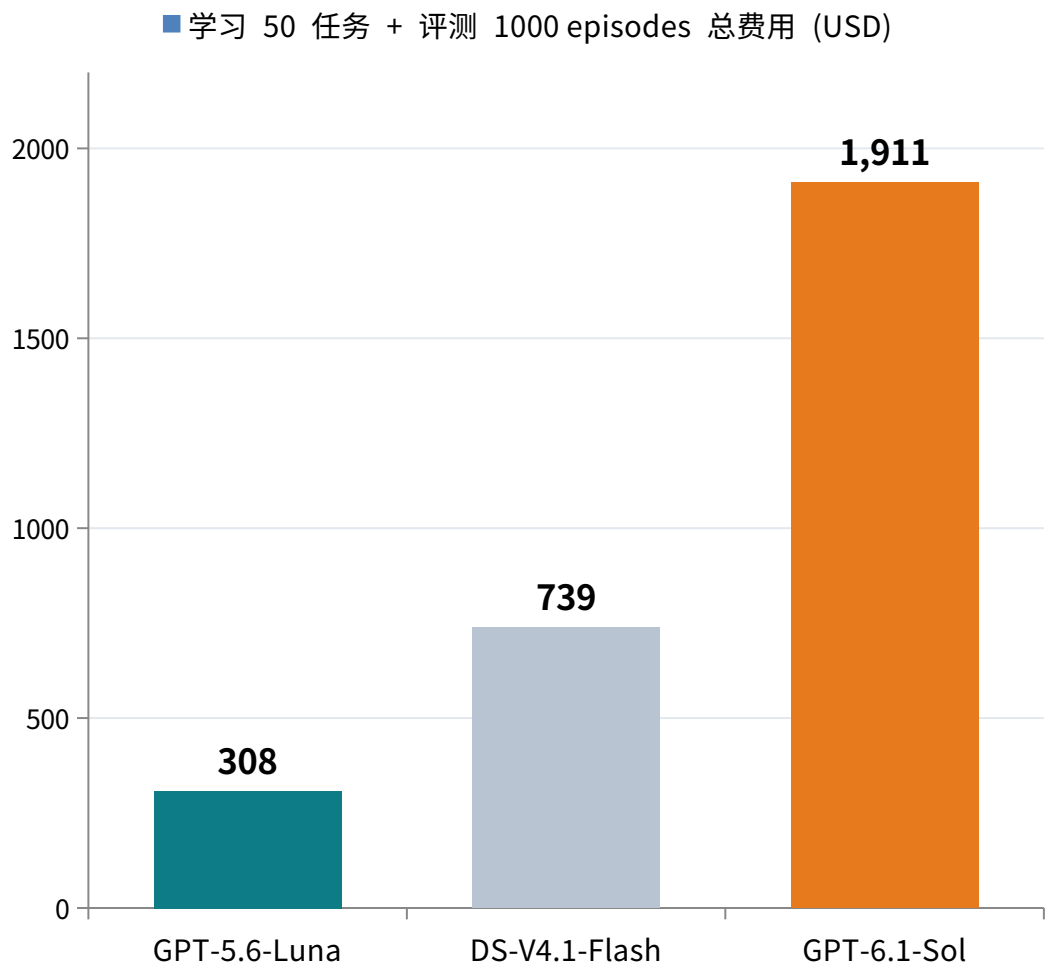
74 → 97 → 112

Guidance 条目数增长
（ active / probation / retired ）

4/10 → 10/10

handover_block： 两条 guidance
在相同 seed 上的效果（ Fig.16 ）

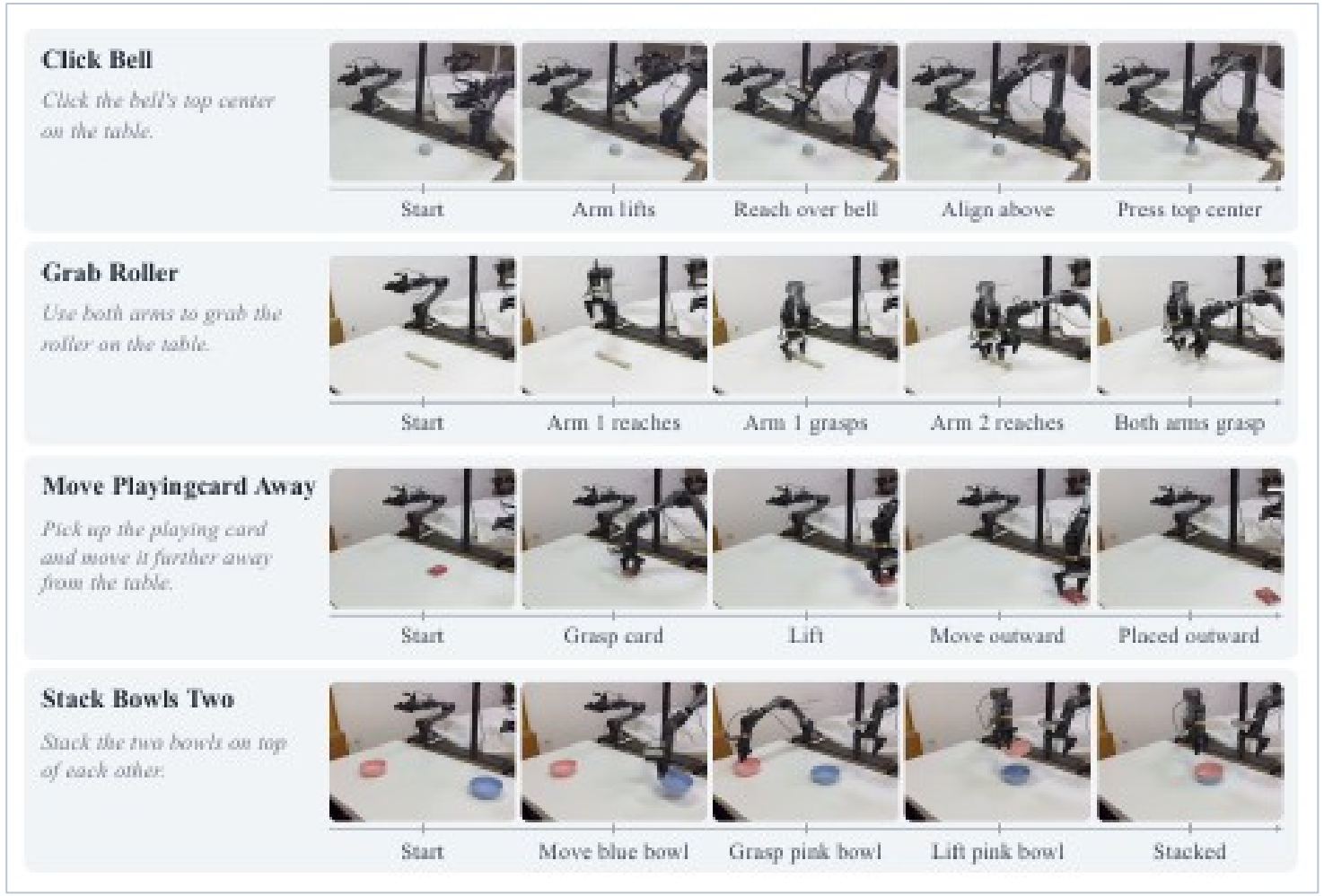
代价：每个 episode 约 55 次模型调用、数百万 token 、数分钟

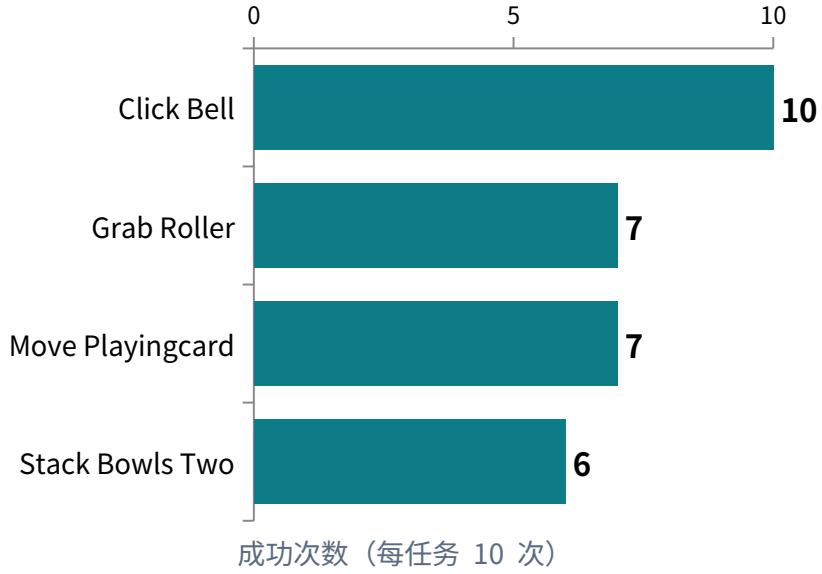


每 episode （评测）	Flash	Luna	Sol
平均成功率 %	78.1	77.9	83.4
模型调用次数	53.1	62.2	54.7
Token （百万）	3.89	2.61	2.33
仿真步数	183	190	149
中位耗时（分钟）	3.6	10.4	6.4
输出 token / 调用	4,750	455	245

- 评估阶段占总费用的大头；约 80% 的时间在等基础模型返回
- Flash 单价低，但每次调用输出约为 Sol 的 19 倍，总价反超 Luna
- 越难的任务越耗 token ；记忆冻结后可重复使用，无额外学习成本

真机：仿真中学到的记忆经轻量适配器迁移到异构双臂 PiPER





怎么做

适配器只改坐标约定、运动学、夹爪参数；workflow 、物体角色、相对规格不变；不重录演示、不重新排练。

注意

离线规划整段动作、无在线重规划；失败主要来自夹爪对位与真实深度误差。

评价：范式清晰、工程严谨，但比较并不完全对等

亮点

- 范式清晰：按“学习信号来自演示还是世界 × 知识存在权重还是上下文”统一 IL / RL / ICL / WICL
- 防记忆设计：排练与测试实例不相交，只计首次尝试
- 数据隔离由系统构造保证；去掉特权监督后成绩不变（86.5 vs 86.0）
- 随机化保持率约 100%，失败归因显示知识类失败不随分布变化
- 记忆是可读文本：可审计、可编辑，自我改进直接作用在文本上

需要谨慎

- 不对等：依赖深度几何感知 + 运动规划器 + 三层 API，环境技能由 coding agent 利用全部 50 任务演示写成；“1 vs 50 demos” 掩盖了这部分先验
- 鲁棒性来源未拆分：深度感知本身对纹理、光照不敏感
- 评测规模：每任务每设置 10 episodes（基线 100）；API 服务质量会使成功率波动数个点
- 慢且贵：单 episode 数分钟、数百万 token
- 真机仅 4 个简单任务、离线规划；可变形物体、紧密双臂协同仍是难点

SUMMARY

Takeaways：有了“世界”在回路中，技能不必只存在权重里

1	**范式**	WICL = ICL + 世界反馈：冻结权重，在可重置、忠实的世界里排练，把语义知识写进上下文。
2	**证据**	RoboTwin 2.0 全 50 任务平均 83.4%（1 demo、零梯度），随机化保持率约 99.5%；三种基础模型都超过训练策略。
3	**机制**	演示提供起点，世界提供纠错；linter 和数据隔离让记忆可以搬到真机上（PiPER 4 任务 6–10/10）。
4	**下一步**	作者提出：在学习型世界模型中排练、自动扩展技能、多智能体协作，以及专门面向具身工具使用训练的基础模型，让 WICL 更快更便宜。